

Phase 3 Documentation

Formal Proof in Lean 4

Chosen Theorem: Expression Determinism

Authors: Ehsan Soveizi & Mahdi Rashidi

Project phase	Phase 3 - Formal Proof
Proof assistant	Lean 4
Chosen theorem	Expression Determinism
Main deliverable	A self-contained proof.lean file and an explanatory report
Core claim	The same expression evaluated in the same state cannot produce two different values.
Report focus	A presentation-ready explanation with examples and proof intuition.

Document purpose

This document explains the formal model, the proof idea, and the Lean 4 theorem in a way that is suitable for an oral presentation. The full source code is intentionally not copied as an appendix; only focused snippets are shown where they help explain the proof.

Contents

- 1. Executive Summary
- 2. Theorem Selection and Formal Meaning
- 3. Minimal Lean Background for Reading the Proof
- 4. Formal Model of Expressions
- 5. Relational Evaluation Rules: EvalExpr
- 6. Proof Strategy and evalExpr_sound
- 7. Main Theorem: expression_determinism
- 8. Statement Semantics Included for Completeness
- 9. Compilation, Limitations, and Presentation Notes

How to use this report

For presentation, start with Sections 1 and 2 to state the goal, use Sections 4 and 5 to explain the model, then use Sections 6 and 7 for the proof idea. Section 8 is included to show that the expression proof is part of a broader operational-semantics model.

1. Executive Summary

Phase 3 moves from testing a generated Promela model to proving a mathematical property about the language model itself. The selected property is Expression Determinism, and it is proved in Lean 4.

The theorem says that expression evaluation has only one possible result. If the same expression is evaluated in the same program state twice, then both evaluations must return the same integer value. This is important because it shows that the expression semantics is unambiguous.

A simple example makes the idea clear. Suppose the state stores $x = 5$ and the expression is $x + 2$. No matter how we derive the evaluation using the formal rules, the result must be 7. The formal theorem generalizes this idea to every expression in the selected Hash subset.

Main idea in one sentence

Every relational evaluation is first shown to agree with the deterministic function `evalExpr`; therefore, two relational evaluations of the same expression must agree with the same function value.

2. Theorem Selection and Formal Meaning

The selected theorem is Expression Determinism. In mathematical notation, the intended statement is:

$$\langle e, s \rangle \text{ evaluates to } v1 \text{ and } \langle e, s \rangle \text{ evaluates to } v2 \text{ implies } v1 = v2$$

Here e is an expression, s is a program state, and $v1$ and $v2$ are integer results. The key point is that both derivations start from exactly the same expression and exactly the same state.

In Lean, the theorem is written using the relational evaluator `EvalExpr`:

```
theorem expression_determinism :
  forall {e : Expr} {s : State} {v1 v2 : Int},
    EvalExpr e s v1 -> EvalExpr e s v2 -> v1 = v2 := by
```

This should be read as: for every expression e , state s , and values $v1$ and $v2$, if `EvalExpr` proves that e evaluates to $v1$ and also proves that e evaluates to $v2$ in the same state, then $v1$ and $v2$ are equal.

3. Minimal Lean Background for Reading the Proof

This section explains a few Lean ideas that appear repeatedly in the code. It is not a full Lean tutorial, but it gives enough context to understand the proof during a presentation.

Lean construct	Meaning in this project
<code>abbrev</code>	Creates a shorter name for a type. For example, <code>Var</code> is just <code>String</code> .
<code>inductive</code>	Defines a new datatype or a relation by listing its constructors/rules.
<code>def</code>	Defines an executable function, such as <code>evalExpr</code> or <code>update</code> .
<code>Prop</code>	The type of logical propositions. <code>EvalExpr e s v</code> is a proposition, not a computed number.
<code>forall</code>	Means “for all”. It introduces universally quantified variables.
<code>-></code>	In theorem statements, it usually means implication: if the left side is proved, the right side follows.
<code>by</code>	Begins tactic-style proof code.
<code>rfl</code>	Closes goals where both sides are definitionally the same.
<code>simp</code>	Simplifies expressions using definitions and known equalities.

A useful mental model is that Lean proofs manipulate evidence. For example, $h : \text{EvalExpr } e \ s \ v$ means h is evidence that expression e evaluates to value v in state s . The proof then studies how this evidence could have been built.

4. Formal Model of Expressions

4.1 Variables and States

```
abbrev Var := String
abbrev State := Var -> Int
```

A variable name is represented as a string. A state is represented as a function from variable names to integer values. This means a state behaves like a memory map: if s is a state, then $s \text{ "x"}$ is the value of x in that state.

Example state

If $s \text{ "x"} = 5$ and $s \text{ "y"} = 10$, then evaluating `Expr.var "x"` in state s returns 5. The state does not need to be stored as a dictionary in the Lean code; it is modeled as a function.

4.2 Expression Syntax

```
inductive Expr where
| num : Int -> Expr
| var : Var -> Expr
| add : Expr -> Expr -> Expr
| mul : Expr -> Expr -> Expr
| lt : Expr -> Expr -> Expr
| eq : Expr -> Expr -> Expr
deriving Repr
```

Constructor	Informal form	Example
<code>Expr.num n</code>	A numeric literal	<code>Expr.num 5</code> represents 5
<code>Expr.var x</code>	A variable expression	<code>Expr.var "x"</code> represents x
<code>Expr.add e1 e2</code>	Addition	$x + 3$
<code>Expr.mul e1 e2</code>	Multiplication	$x * 2$
<code>Expr.lt e1 e2</code>	Less-than comparison	$x < 10$ returns 1 or 0
<code>Expr.eq e1 e2</code>	Equality comparison	$x = 5$ returns 1 or 0

The design represents boolean results as integers. A true comparison is encoded as 1, and a false comparison is encoded as 0. This keeps all expression results in one type, `Int`, and avoids introducing a separate `Value` type.

4.3 Executable Evaluator: `evalExpr`

The function `evalExpr` is the executable meaning of expressions. It takes an expression and a state, then returns the integer value of the expression in that state.

```
def evalExpr (e : Expr) (s : State) : Int :=
  match e with
  | Expr.num n => n
  | Expr.var x => s x
  | Expr.add e1 e2 => evalExpr e1 s + evalExpr e2 s
  | Expr.mul e1 e2 => evalExpr e1 s * evalExpr e2 s
  | Expr.lt e1 e2 => if evalExpr e1 s < evalExpr e2 s then 1 else 0
  | Expr.eq e1 e2 => if evalExpr e1 s = evalExpr e2 s then 1 else 0
```

The `add` and `mul` cases are recursive: Lean first evaluates the smaller subexpressions $e1$ and $e2$, then combines their results. This is structural recursion over the expression tree.

Worked example

Let $e = (x + 3) * 2$ and suppose $s \text{ "x"} = 4$. First, `evalExpr` evaluates x to 4 and 3 to 3. Then $x + 3$ becomes 7. Finally, $7 * 2$ becomes 14. Because `evalExpr` is a function, this result is unique.

5. Relational Evaluation Rules: EvalExpr

EvalExpr is not a function. It is a logical relation. The proposition $\text{EvalExpr } e \ s \ v$ means that expression e evaluates to value v in state s . Each constructor of EvalExpr is a formal rule for building evidence of evaluation.

```
inductive EvalExpr : Expr -> State -> Int -> Prop where
```

Function versus relation

evalExpr $e \ s$ computes a number. EvalExpr $e \ s \ v$ is a proposition saying that v is a valid result. The determinism theorem proves that this relation cannot assign two different values to the same expression in the same state.

5.1 Literals and Variables

```
| num : forall (n : Int) (s : State),
  EvalExpr (Expr.num n) s n

| var : forall (x : Var) (s : State),
  EvalExpr (Expr.var x) s (s x)
```

The num rule says that a literal number evaluates to itself in any state. The var rule says that a variable evaluates to the value stored for that variable in the current state. For example, if $s \ "x" = 5$, then EvalExpr (Expr.var "x") $s \ 5$ is valid.

5.2 Compound Expressions

```
| add : forall (e1 e2 : Expr) (s : State) (v1 v2 : Int),
  EvalExpr e1 s v1 ->
  EvalExpr e2 s v2 ->
  EvalExpr (Expr.add e1 e2) s (v1 + v2)
```

The add rule is compositional. It first needs evidence that $e1$ evaluates to $v1$ and evidence that $e2$ evaluates to $v2$. Only then can it conclude that $e1 + e2$ evaluates to $v1 + v2$.

The mul rule has the same structure, except the final result is $v1 * v2$. The comparison rules lt and eq also evaluate both sides first, then return 1 or 0 depending on the comparison.

```
| lt : forall (e1 e2 : Expr) (s : State) (v1 v2 : Int),
  EvalExpr e1 s v1 ->
  EvalExpr e2 s v2 ->
  EvalExpr (Expr.lt e1 e2) s (if v1 < v2 then 1 else 0)
```

Presentation example

If e is $x < 10$ and $s \ "x" = 4$, the relation first derives EvalExpr (Expr.var "x") $s \ 4$ and EvalExpr (Expr.num 10) $s \ 10$. Since $4 < 10$, the final result is 1.

6. Proof Strategy and evalExpr_sound

The direct way to prove expression determinism would be to compare two EvalExpr derivations case by case. That works, but it produces a longer proof. This implementation uses a cleaner strategy: first connect the relation EvalExpr to the deterministic function evalExpr.

```
theorem evalExpr_sound :
  forall {e : Expr} {s : State} {v : Int},
    EvalExpr e s v -> v = evalExpr e s := by
```

This theorem says that any value produced by the relational rules is exactly the same value produced by the executable evaluator. It is called soundness here because the relation does not produce results outside the reference evaluator.

The proof uses induction on the derivation $h : \text{EvalExpr } e \ s \ v$. This means Lean checks each possible rule that could have built h : num, var, add, mul, lt, or eq.

Case	Why it closes
num	The relational result is n , and $\text{evalExpr } (\text{Expr.num } n) \ s$ also reduces to n . rfl proves $n = n$.
var	The relational result is $s \ x$, and $\text{evalExpr } (\text{Expr.var } x) \ s$ also reduces to $s \ x$. rfl proves equality.
add	The induction hypotheses rewrite $v1$ and $v2$ into $\text{evalExpr } e1 \ s$ and $\text{evalExpr } e2 \ s$. simp finishes the arithmetic-shaped equality.
mul	Same idea as add, but the final operation is multiplication.
lt	The induction hypotheses make both comparison inputs identical; simp reduces the if-expression.
eq	Same idea as lt, but with equality comparison.

Key intuition

The proof does not claim that EvalExpr is deterministic directly. Instead, it proves that EvalExpr always agrees with evalExpr . Since evalExpr is an ordinary function, it already has exactly one result.

7. Main Theorem: expression_determinism

```
theorem expression_determinism :
  forall {e : Expr} {s : State} {v1 v2 : Int},
    EvalExpr e s v1 -> EvalExpr e s v2 -> v1 = v2 := by
  intro e s v1 v2 hEval1 hEval2
  have h1 : v1 = evalExpr e s := evalExpr_sound hEval1
  have h2 : v2 = evalExpr e s := evalExpr_sound hEval2
  exact Eq.trans h1 h2.symm
```

The proof receives two derivations. hEval1 says that e evaluates to $v1$ in state s . hEval2 says that the same expression e evaluates to $v2$ in the same state s .

1. Apply evalExpr_sound to hEval1 . This gives $\text{h1} : v1 = \text{evalExpr } e \ s$.
2. Apply evalExpr_sound to hEval2 . This gives $\text{h2} : v2 = \text{evalExpr } e \ s$.
3. Reverse h2 using h2.symm , giving $\text{evalExpr } e \ s = v2$.
4. Use equality transitivity: $v1 = \text{evalExpr } e \ s$ and $\text{evalExpr } e \ s = v2$ imply $v1 = v2$.

```
v1 = evalExpr e s
evalExpr e s = v2
-----
v1 = v2
```

Why the final proof is short

The structural work is done in evalExpr_sound . The main theorem only connects two derivations through the same deterministic reference value.

8. Statement Semantics Included for Completeness

Although the selected theorem is about expression evaluation, the Lean file also includes statement syntax and statement-level big-step semantics. This shows that the expression proof is part of a broader operational-semantics model for the selected Hash subset.

8.1 Statement Syntax and Update

```
inductive Stmt where
```

```

| assign : Var -> Expr -> Stmt
| seq   : Stmt -> Stmt -> Stmt
| ifElse : Expr -> Stmt -> Stmt -> Stmt
| while  : Expr -> Stmt -> Stmt

```

Stmt represents assignments, sequences, conditional statements, and while loops. Assignment uses the update function to produce a new state.

```

def update (s : State) (x : Var) (v : Int) : State :=
  fun y => if y == x then v else s y

```

The variable `y` is the variable being queried in the new state. If `y` is the updated variable `x`, the new state returns `v`. Otherwise, it keeps the old value `s y`. For example, `(update s "x" 100) "x"` returns 100, while `(update s "x" 100) "z"` returns the old value `s "z"`.

8.2 BigStep Statement Rules

Rule	Meaning
assign	Evaluate the right-hand expression, then update the assigned variable in the state.
seq	Run the first statement to an intermediate state, then run the second statement from that state.
ifTrue	If the condition evaluates to a nonzero value, run the then-branch.
ifFalse	If the condition evaluates to 0, run the else-branch.
whileFalse	If the loop condition evaluates to 0 at the beginning, stop immediately and keep the state unchanged.
whileTrue	If the loop condition is nonzero, run the body once and then run the same while loop again from the intermediate state.

The repetition logic of while is expressed recursively: one iteration of the body is followed by another BigStep derivation of the same while statement from the new state. The false rule is the stopping case.

forLoop as syntactic sugar

The Hash form `baraye(init; cond; step) { body }` can be represented as `init; while(cond) { body; step }`. Therefore, no separate proof rule is required for `forLoop`.

9. Compilation, Limitations, and Presentation Notes

9.1 Compilation

The Lean file is self-contained and does not require Mathlib or external imports. It can be checked with:

```
lean proof.lean
```

On success, Lean usually prints no output. This silence is expected: it means all definitions and theorems were accepted. If there is a syntax error, a missing definition, or an invalid proof step, Lean reports an error.

9.2 Limitations and Extensions

- The selected theorem is expression determinism, not full statement determinism.
- Boolean results are encoded as Int values, with `true = 1` and `false = 0`.
- The model does not include division, exceptions, `break/continue`, arrays, or I/O.
- A possible extension is to prove determinism for BigStep statements or prove a loop invariant theorem.

9.3 Presentation Script

Short oral explanation

In Phase 3, we modeled a subset of Hash expressions in Lean 4. We defined both an executable evaluator, `evalExpr`, and a relational big-step evaluator, `EvalExpr`. Then we proved `evalExpr_sound`, which states that every relational evaluation agrees with `evalExpr`. Finally, we used that result to prove Expression Determinism: if the same expression is evaluated twice in the same state, both evaluations must produce the same value.

Possible question	Suggested answer
Why use both <code>evalExpr</code> and <code>EvalExpr</code> ?	<code>evalExpr</code> is convenient as a deterministic reference function; <code>EvalExpr</code> is the formal relational semantics. The soundness theorem connects them.
Why is the main theorem short?	Because <code>evalExpr_sound</code> already proves that every relational result equals the function result.
Why include <code>Stmt</code> and <code>BigStep</code> if the theorem is about expressions?	They complete the operational-semantics model for the selected Hash subset and show how expressions are used inside statements.